LINUX INFRASTRUCTURE OBSERVABILITY DASHBOARD PROJECT: TUI DASHBOARD

JONAS RADOVANOVIC, JORDAN DEXTER, ADRIAN HERNANDEZ VEGA

- **Project Link**: https://github.com/gaspar-of-time/LIOD-Project-UNIX
- **Section:** COP3604 Unix Administration

Date: October 16, 2026.

1. Introduction & Objectives

Our TUI Dashboard project addresses the task of creating a client dashboard to graphically monitor several select system statistics of multiple connected servers. Each server has an API running as a system service that collects critical system statistics, including transferring/receiving bytes and swapping, CPU, storage, and memory utilization. After gathering information, when the client requests, the API sends these statistics over a HTTP connection. The client handles receiving this information and displaying it in a custom dashboard on a graph, with controls for when an alert should be shown if server statistics are within a specified range. The main objectives were to deploy a server on Google Cloud, keep the API running with systemd, simulate load with stress-ng, restrict API access to a single IP address, and have the API restart automatically if it fails.

2. System Architecture

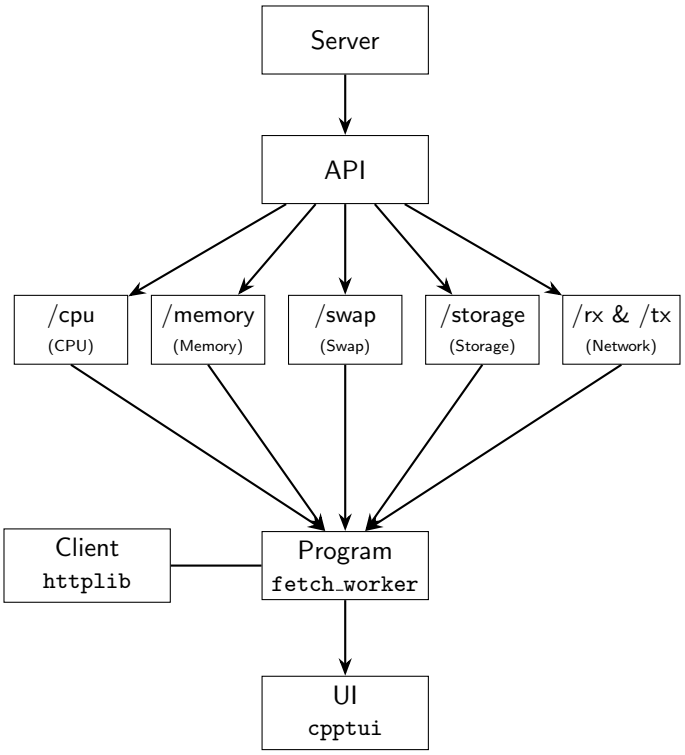
Languages used:

- C++ (89.5%)
- Shell (9.1%)
- Makefile (1.4%)

FOSS Modules:

- **cpp-httplib**: For sending (serverMain.cpp)
- **cpp-sys-stats**: Grabbing statisics
- **cpp-tui**: UI elements, charts, and graphs (clientMain.cpp)

2.1. **Diagram.**



Each server runs the API as a systemd service so it starts on boot and restarts if it crashes. A stress-ng script runs on each server to generate load. The API is restricted to a limited set of addresses on port 7777. The client polls each server's API and displays the results in the dashboard.

2.2. **Failure Handling and Security.** A cron job runs every 5 minutes to check that the API is responding. If the check fails, it restarts the API service.

2.3. **Deployment.** Three servers were set up on Google Cloud Platform and report to one dashboard.

3. Analysis & Results

To evaluate the system under load, the server and stress test were executed simultaneously within the Codespace environment. The following command sequence was used to initiate the server daemon and the stress script:

```
./bin/LIODd
./scripts/stress_server.sh
```

The execution of these commands in the terminal is shown below:

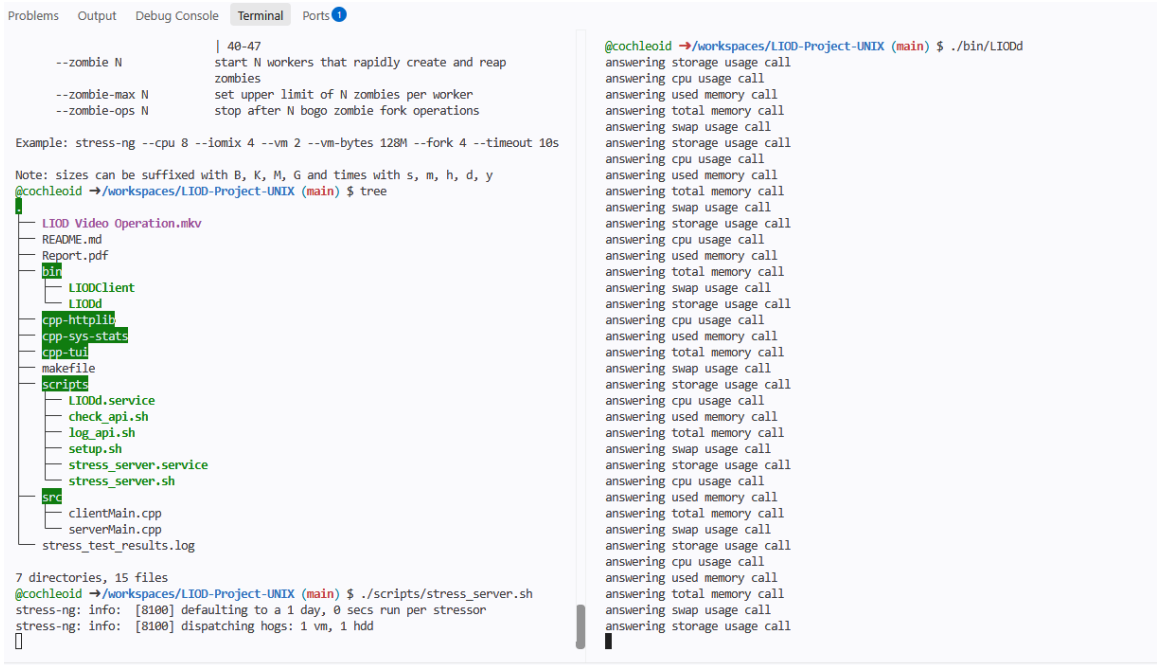


Figure 1. Terminal output displaying the server and stress command execution

Following the initiation of the stress test, the client dashboard was launched to monitor the server's metrics. As the simulated load increased, the CPU utilization rapidly climbed until it reached 100.000000%. Upon hitting this maximum capacity, the dashboard successfully triggered the threshold alert, promptly displaying a warning notification in the bottom right corner stating "Server 1's CPU is 100.000000%". Visual confirmation of this behavior is captured in the dashboard interface below:

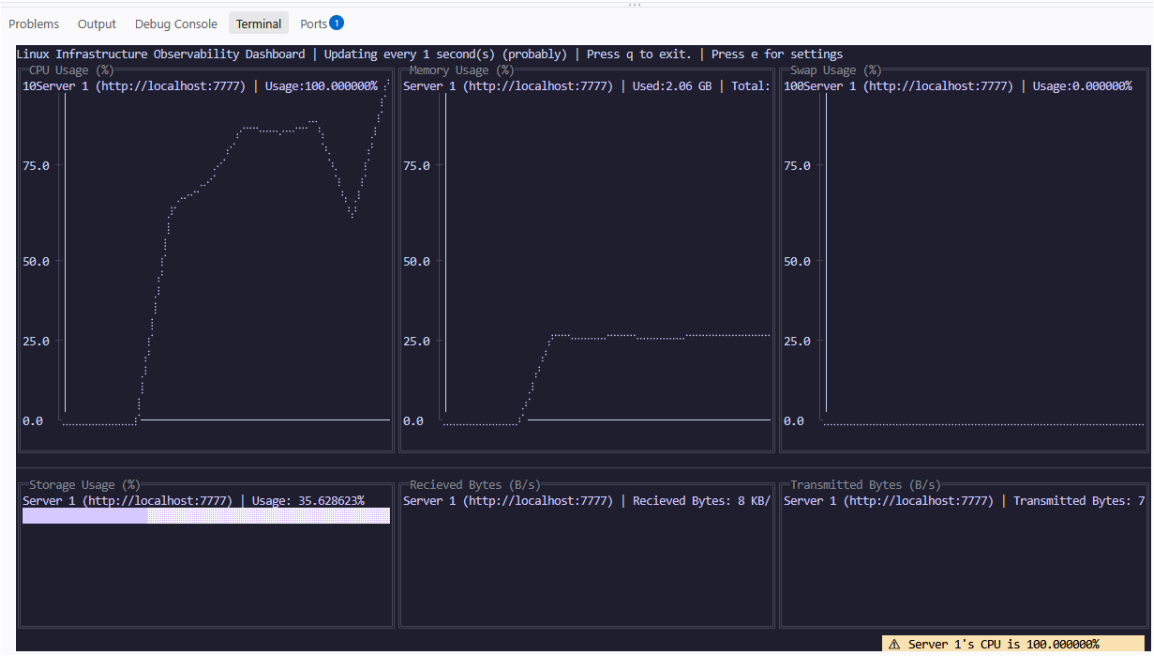


FIGURE 2. TUI Dashboard showing 100% CPU usage and the corresponding alert notification

4. PROBLEMS FACED

The biggest challenge was using *cpp-tui* to display all three graphs simultaneously with a livestream of data. The first iteration, which used a fork of a *cpp-tui* dev sample, would only display bar charts instead of the dynamic line graphs. We eventually switched to the line chart component and rebuilt the layout, which is what the final version uses.

As shown in the screenshots below (Jonas), being able to capture all the information in the dashboard took a few iterations and drafts:

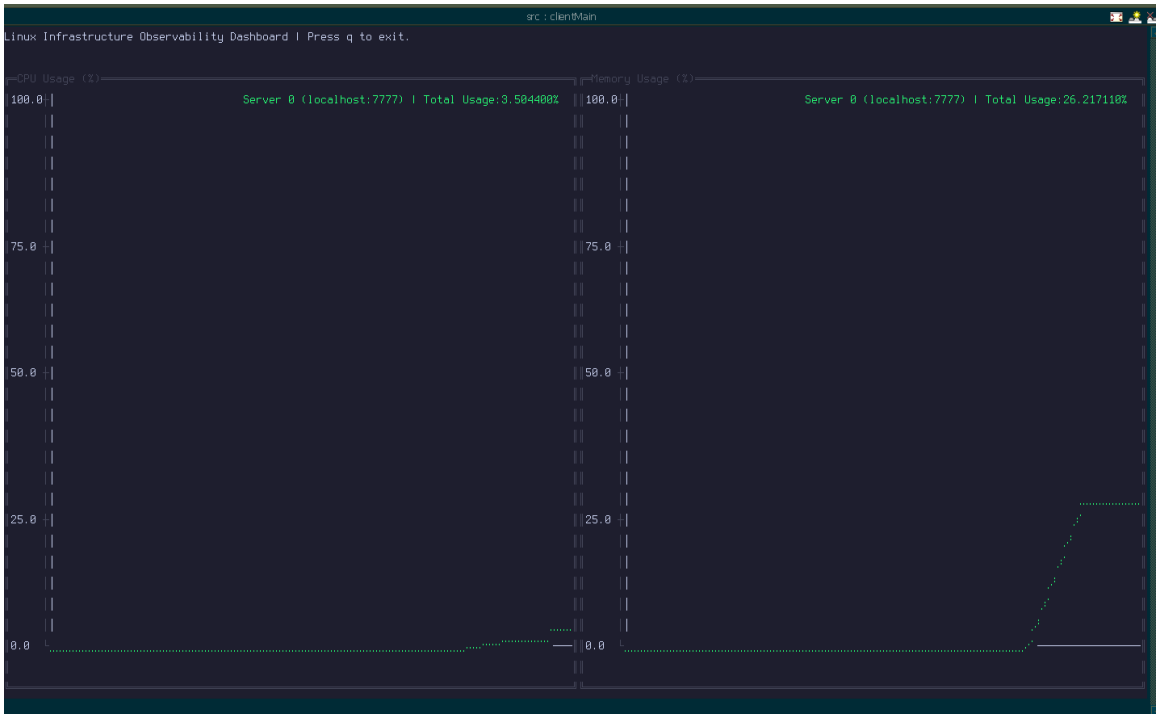


FIGURE 3. First iteration of the dashboard

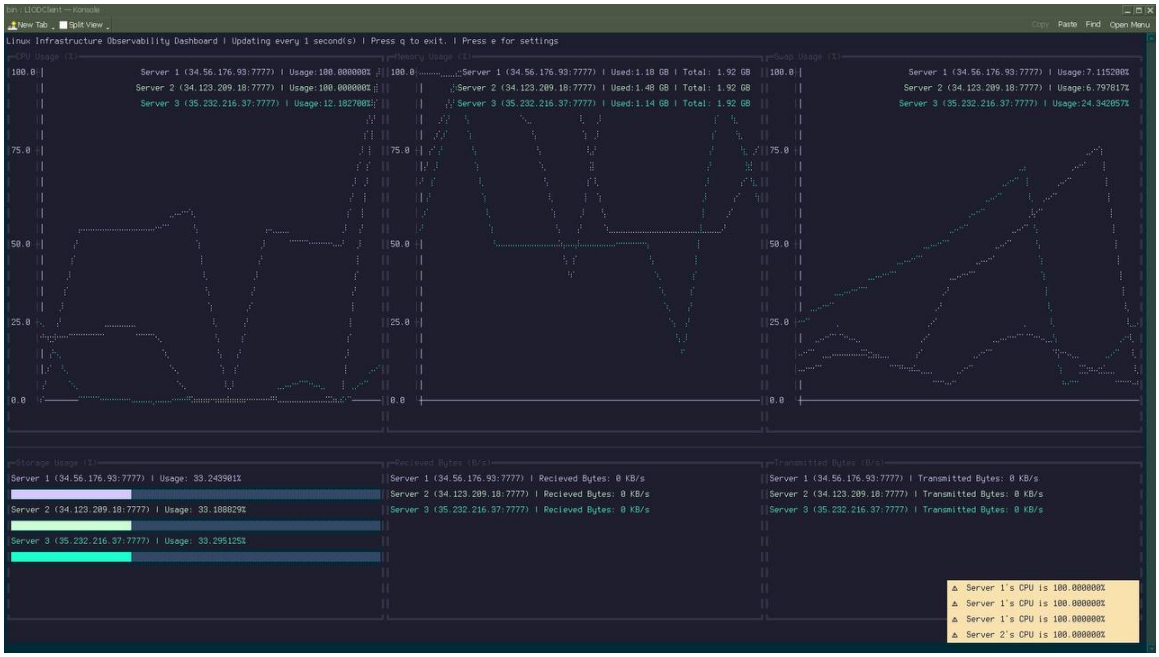


FIGURE 4. Second iteration of the dashboard, making better use of the screen real estate

5. Reflection

5.1. **Reflection 1: Adrian.** I hadn't ever done a "formal" report for computer science work, but knowing how to work LaTeX from math classes came in handy (not to mention, a tricky diagram.) Having to document steps and architecture wasn't something I was comfortable with as first, since the author has to survey the code well (sometimes prompting your members) to write at different levels of abstraction. While I mainly worked on the report, I gave the UI a shot, but it wasn't fully compliant with the specifications.

5.2. **Reflection 2: Jonas.** I learned a couple different things during the course of this project: it's difficult to implement raw TCP communication in C++ and it's difficult to figure out how to create a program that can handle an undefined number of inputs. Initially, when writing the API for the server, I attempted to use raw TCP communication to request and send data between server and client. From this attempt, I learned that TCP communication is unreliable when sending, where it doesn't always send the same amount of data, and that this needs to be accounted for when programming. This unreliability and the fact that my initial plan for the API required the server and client to be synchronized ultimately made me move towards an HTTP API library for the final implementation. When I was rewriting the UI to better meet the requirements for the project, I wanted to make the client support monitoring an undefined number of servers rather than the 3 that were required. After a bit of thinking, I decided on using vectors to store the server information, where each server had an index for its data in vectors for each recorded metric. This was chosen since I needed to use vectors anyway for storing the data for the line charts for each server, although in hindsight arrays would have worked just as well, since the number of servers does not change after initialization.

5.3. **Reflection 3: Jordan.** This was my first time working with a collaborative project through Github. Overall, it went smoothly, with most of the difficulty coming from learning how to code for the project rather than familiarizing myself with Github. The code that others had written were dotted with an adequate amount of comments, and each of the Github commits were properly named and described, so parsing code and knowing what others were up to was not difficult. Most of our communication was in-person and over Discord, which enabled good coordination and task distribution.

6. Contributions

Member	Activity	Effort
Jonas	Main programming, fully wrote server	44%
Adrian	Main report creation and formatting, first draft of client UI	28%
Jordan	Cron job, .system files, report writing and revisions	28%